Labo 14 — Réponses commentées

Chaque réponse suit le même schéma : la réponse, le mécanisme, la nuance ou le piège, un exemple vérifiable au terminal.

Question 1 — Du labo 08 au cluster

Réponse. `ENV` → reste dans l'image (une valeur par défaut, inchangée). `-e` → `env:` avec une `value` en dur dans le template. `--env-file api.env` → une ConfigMap (`--from-env-file`) injectée par `envFrom`. `application.yaml` en bind mount → une ConfigMap `--from-file` montée en volume dans `/config/`. `podman secret` + `_FILE` → un Secret monté en volume `secret:` avec `DB_PASSWORD_FILE`. Sans équivalent : le `-e` **tapé sur la ligne de commande** — et c'est une bonne chose : dans le cluster, toute configuration est un objet versionné et relisible, pas un argument perdu dans l'historique du shell.

Pourquoi. Le cluster n'a pas de ligne de commande : chaque valeur doit exister dans l'API pour être appliquée sur n'importe quel nœud.

Nuance. `kubectl set env deployment/api APP_PROFILE=prod` existe et modifie le template en dur — pratique pour un test, à proscrire en production (la valeur n'est plus dans Git).

Exemple.

```
kubectl create configmap api-config --from-env-file=api.env --dry-run=client -o yaml
```

Question 2 — Variables figées, fichiers vivants

Réponse. Premier `grep` : `APP_MESSAGE=bonjour depuis une ConfigMap`, **l'ancienne valeur** — les variables sont copiées dans le processus au démarrage. Second `cat` (après une minute) : `<h1>modifie</h1>` — le volume est mis à jour par le kubelet. Pour que l'API voie le nouveau message : `kubectl rollout restart deployment/api`, qui recrée les Pods.

Pourquoi. Une variable d'environnement appartient au processus ; un volume ConfigMap est un répertoire que le kubelet resynchronise périodiquement (lien symbolique `..data` basculé atomiquement).

Nuance. Le délai de mise à jour du volume dépend de la période de synchronisation du kubelet (une minute par défaut) et du cache ; comptez une à deux minutes. Et nginx, lui, a chargé `default.conf` au démarrage : un nouveau fichier de configuration exige un `nginx -s reload`, un nouvel `index.html` est servi immédiatement.

Exemple.

```
kubectl exec deploy/web -- ls -l /usr/share/nginx/html/     # index.html -> ..data/index.html
kubectl rollout restart deployment/api && kubectl exec deploy/api -- env | grep APP_MESSAGE
```

Question 3 — `CreateContainerConfigError`

Réponse. `kubectl describe pod api-5dff955d5b-rz7sd` montre dans les Events : `Error: couldn't find key profil in ConfigMap lab14/api-config` — la clé référencée n'existe pas (`profil` au lieu de `APP_PROFILE`). Le kubelet ne peut pas construire l'environnement du conteneur, donc ne le démarre pas. L'ancien Pod tourne encore parce que le rollout attend que le nouveau soit prêt

avant de retirer l'ancien (labo 12) : le service n'est pas touché. Correction : réparer la clé (ou `rollout undo`).

Pourquoi. Les références à ConfigMaps et Secrets sont résolues **à la création du conteneur**, pas à l'`apply` : un manifeste peut être accepté et échouer plus tard.

Nuance. `optional: true` sur la référence (`configMapKeyRef` ou `configMapRef`) fait ignorer une clé ou un objet absent au lieu de bloquer — à n'utiliser que pour une configuration réellement facultative.

Exemple.

```
kubectl describe pod -l app=api | grep -A1 "couldn't find key"
kubectl rollout undo deployment/api
```

Question 4 — « Chiffrés, donc sûrs »

Réponse. `kubectl get secret db-secret -o jsonpath='{.data.POSTGRES_PASSWORD}' | base64 -d` affiche `S3cr3t-pg` : le base64 est un **encodage**, réversible par n'importe qui. Ce qu'un Secret apporte réellement : (1) une **classe d'objet distincte**, que le RBAC peut interdire en lecture séparément des ConfigMaps ; (2) un montage en **tmpfs** sur le nœud, jamais écrit sur son disque ; (3) le **masquage** dans `describe` et les outils, qui évite la fuite par copier-coller. Les trois mesures d'entreprise : chiffrement au repos dans etcd (`EncryptionConfiguration` ou KMS), RBAC strict sur `get`/`list secrets`, et aucun Secret en clair dans Git (External Secrets, Sealed Secrets, SOPS).

Pourquoi. Le base64 n'existe que pour transporter des octets arbitraires dans du JSON/YAML ; il n'a jamais eu de rôle de protection.

Nuance. Même chiffré au repos, un Secret est **en clair** pour quiconque a le droit de le lire via l'API : le RBAC est la vraie frontière.

Exemple.

```
kubectl get secret db-secret -o yaml | grep POSTGRES_PASSWORD     # UzNjcjN0LXBn
echo UzNjcjN0LXBn | base64 -d                                      # S3cr3t-pg
```

Question 5 — Variable ou fichier pour le mot de passe

Réponse. Visibilité : en variable, `kubectl exec deploy/api -- env` affiche le mot de passe (et tout `dump` d'environnement aussi) ; en fichier, seul le chemin est visible, la valeur exige d'ouvrir le fichier. Rotation : en variable, un nouveau Secret n'est vu qu'après `rollout restart` ; en fichier, le kubelet met le fichier à jour en une minute, et une application qui relit le fichier (ou un pool qui se reconnecte avec le nouveau mot de passe) suit sans redémarrage. Spring Boot : les deux fonctionnent, la seconde via `spring.config.import=optional:configtree:/run/secrets/db/`. On choisit le **fichier**, comme au labo 08.

Pourquoi. L'environnement est une métadonnée largement exposée ; un fichier monté n'est accessible qu'au processus.

Nuance. Beaucoup d'images officielles n'acceptent que la variable (`POSTGRES_PASSWORD`) **ou** le fichier (`POSTGRES_PASSWORD_FILE`) : vérifier la documentation de l'image.

Exemple.

```
kubectl exec deploy/api -- env | grep -c S3cr3t        # 0 avec le volume
kubectl exec deploy/api -- cat /run/secrets/db/POSTGRES_PASSWORD
```

Question 6 — `subPath` ne se met pas à jour

Réponse. Avec `subPath`, le kubelet **copie** le fichier dans le conteneur au démarrage au lieu de monter le répertoire à liens symboliques : les mises à jour de la ConfigMap ne sont jamais propagées. Deux issues : (1) monter la ConfigMap sur un **répertoire dédié** (`/etc/nginx/conf.d/` entier, ou `/etc/nginx/extra/` inclus depuis la configuration principale) sans `subPath` ; (2) accepter le `subPath` et déclencher un `rollout restart` à chaque changement — ou changer le nom de la ConfigMap (question 12), ce qui modifie le template et provoque le rollout.

Pourquoi. `subPath` existe pour injecter un fichier dans un répertoire qui contient d'autres choses ; le prix est la perte de la mise à jour dynamique.

Nuance. Le même piège vaut pour les Secrets montés avec `subPath` : la rotation ne se voit pas.

Exemple.

```
kubectl get pod -l app=web -o jsonpath='{.items[0].spec.containers[0].volumeMounts}' | grep -o 'subPath[^}]*'
```

Question 7 — Même Deployment, autre namespace

Réponse. Le Deployment est créé dans `prod`, ses Pods démarrent avec `APP_PROFILE=prod` et le mot de passe de production : les références se résolvent **dans le namespace du Pod**, vers les objets `api-config` et `db-secret` qui s'y trouvent. Rien dans le manifeste n'a changé. Cela démontre que la promotion d'une image (labo 10) est un `apply` du même fichier dans un autre namespace (ou cluster), où la configuration est différente : l'artefact testé est l'artefact déployé.

Pourquoi. Les références par nom sont locales au namespace ; c'est la frontière qui porte la configuration.

Nuance. Si `api-config` manque dans `prod`, les Pods restent en `CreateContainerConfigError` (question 3) : la ConfigMap fait partie de ce qu'il faut déployer, et Helm/Kustomize (labo 17) l'embarquent avec le reste.

Exemple.

```
kubectl apply -f deployment-api.yaml -n prod && kubectl exec -n prod deploy/api -- env | grep APP_PROFILE   # prod
```

Question 8 — `--dry-run=client -o yaml`

Réponse. La commande **n'envoie rien** au cluster : elle imprime le manifeste `Secret` complet, avec `data.password` déjà encodé en base64 (`eA==` pour `x`), prêt à être enregistré dans un fichier. On l'utilise pour obtenir la structure exacte sans se tromper d'encodage ni d'indentation. Ce qu'il ne faut pas faire : commiter ce fichier tel quel — il contient le secret, base64 ou pas. Il doit être chiffré (SOPS, Sealed Secrets) avant d'entrer dans Git, ou servir de modèle sans valeur.

Pourquoi. `--dry-run=client` fait faire par `kubectl` tout le travail de construction de l'objet, sans appel à l'API.

Nuance. `--dry-run=server` envoie l'objet pour validation sans le persister : utile pour tester les webhooks et les règles du cluster.

Exemple.

```
kubectl create secret generic db --from-literal=password=x --dry-run=client -o yaml | grep -A1
'^data:'   # password: eA==
```

Question 9 — Secret collé dans un ticket

Réponse. (1) Le Secret est **compromis** : base64 ou pas, il a été lu. (2) **Rotation** : nouveau mot de passe côté base, `kubectl apply` (ou le gestionnaire de secrets) d'un `db-secret` mis à jour. (3) **Propagation** : si le Secret est monté en volume, les Pods voient le nouveau fichier en une minute, et une application qui relit le fichier suit ; en variable, `rollout restart deployment/api` est obligatoire. (4) Révocation de l'ancien mot de passe, vérification des journaux de la base, nettoyage du ticket en dernier. Ce qui rend la rotation rapide : un Secret référencé par nom, monté en fichier, lu par une application qui sait se reconnecter ; lente : une valeur recopiée dans plusieurs manifestes, en variable, avec des Pods à redémarrer un par un.

Pourquoi. La seule réponse sûre reste de rendre la valeur fuitée inutile.

Nuance. Le RBAC du labo 16 réduit le nombre de personnes capables de produire ce `get -o yaml` : c'est la mesure préventive.

Exemple.

```
kubectl create secret generic db-secret --from-literal=POSTGRES_PASSWORD=nouveau --dry-
run=client -o yaml | kubectl apply -f -
kubectl rollout restart deployment/api
```

Question 10 — Un répertoire vide

Réponse. Le **`volumeMounts`** manque (ou son `mountPath` ne vise pas `/usr/share/nginx/html`) : la ConfigMap est peut-être déclarée dans `volumes:`, mais rien ne la monte dans le conteneur — ou le volume est monté à un autre chemin. Vérification : `kubectl get pod -l app=web -o yaml` et comparer `spec.volumes` (le volume `html` avec `configMap: web-html`) et `spec.containers[0].volumeMounts` (un montage `html` sur `/usr/share/nginx/html`). Un répertoire vide (et non le `index.html` d'origine de l'image) indique qu'un volume **est** monté là, mais pas le bon : par exemple une ConfigMap vide ou d'un autre nom.

Pourquoi. `volumes` déclare, `volumeMounts` branche ; les deux sont nécessaires, et un montage masque ce que l'image contenait.

Nuance. `kubectl describe pod` montre `Mounts:` et `Volumes:` lisiblement, et un volume dont la ConfigMap n'existe pas bloque en `ContainerCreating` avec `configmap "web-html" not found`.

Exemple.

```
kubectl get pod -l app=web -o jsonpath='{.items[0].spec.containers[0].volumeMounts}' ; echo
kubectl get pod -l app=web -o jsonpath='{.items[0].spec.volumes}' ; echo
```

Question 11 — Monter tout `/etc/nginx/conf.d`

Réponse. Monter le répertoire entier **masque** le contenu de l'image à cet endroit : le `default.conf` d'origine disparaît, remplacé par le seul fichier de la ConfigMap — c'est voulu, puisque c'est ce fichier qu'on remplace, et cela garde la mise à jour dynamique (pas de `subPath`). Pour ajouter un second fichier `.conf` : le mettre dans la **même** ConfigMap (une clé de plus, un fichier de plus dans le répertoire), ou utiliser `projected` pour combiner plusieurs ConfigMaps dans un même montage.

Pourquoi. Un montage est un remplacement de répertoire, exactement comme un bind mount du labo 06 ; l'image en dessous n'est pas fusionnée.

Nuance. Si l'image attendait d'autres fichiers dans ce répertoire (ici non), il faudrait les reproduire dans la ConfigMap ou monter ailleurs.

Exemple.

```
kubectl exec deploy/web -- ls /etc/nginx/conf.d/     # default.conf seulement, venant de la ConfigMap
```

Question 12 — `immutable` ou nom versionné

Réponse. `immutable: true` interdit toute modification de l'objet : protection contre une édition accidentelle en production, et le kubelet cesse de le surveiller (moins de charge sur l'API server) — mais changer la configuration impose de **créer un autre objet** et de changer la référence. Nommer `api-config-v2` et pointer le Deployment dessus change le **template**, donc déclenche un **rollout** : les Pods redémarrent avec la nouvelle configuration, et `rollout undo` ramène l'ancienne (l'ancien ReplicaSet référence `api-config-v1`). C'est pourquoi Kustomize (`configMapGenerator`) et Helm (annotation de checksum) ajoutent un suffixe de hachage : toute modification de la configuration devient une révision du Deployment, traçable et réversible.

Pourquoi. Une ConfigMap modifiée en place n'est pas une révision ; une ConfigMap renommée l'est.

Nuance. Les anciennes ConfigMaps suffixées s'accumulent ; Kustomize et Helm les nettoient (ou non) selon la configuration.

Exemple.

```
kubectl patch configmap api-config -p '{"immutable":true}' && kubectl patch configmap api-config -p '{"data":{"APP_PROFILE":"x"}}'   # field is immutable
```

Question 13 — Aucun Secret dans Git

Réponse. Le dépôt contient le Deployment (qui référence `db-secret` par nom) et un objet **ExternalSecret** : « crée un Secret `db-secret` avec la clé `POSTGRES_PASSWORD` prise dans Vault au chemin `secret/app/prod/db` ». Aucune valeur. Dans le cluster, l'**External Secrets Operator** lit cet objet, s'authentifie auprès du gestionnaire de secrets (Vault, AWS Secrets Manager…), récupère la valeur et **crée ou met à jour** le Secret Kubernetes — et le rafraîchit périodiquement, ce qui fait la rotation. Avec Sealed Secrets, le dépôt contient le Secret **chiffré** avec la clé publique du cluster, et un contrôleur le déchiffre en Secret ordinaire. Le lien avec le labo 08 : le gestionnaire de secrets est la source de vérité, le Secret Kubernetes n'est qu'un cache local injecté dans les Pods.

Pourquoi. Git est un support de diffusion ; le cluster est l'endroit où un secret peut vivre sous contrôle d'accès.

Nuance. Les deux approches gardent le Secret **en clair dans l'API** une fois créé : le chiffrement au repos et le RBAC restent nécessaires.

Exemple.

```
kind: ExternalSecret
spec:
  secretStoreRef: { name: vault }
  target: { name: db-secret }
  data:
    - secretKey: POSTGRES_PASSWORD
      remoteRef: { key: secret/app/prod/db, property: password }
```